



UNIDADE IV

Engenharia de Software Ágil Aplicada

Prof. Me. Edson Moreno

Introdução

- A unidade IV aborda de forma integrada a qualidade do software ao longo do ciclo de vida, combinando Verificação & Validação (V&V) com gestão de mudanças e evolução.
- Inicialmente, são apresentados os princípios de V&V, destacando a verificação de código, a depuração de falhas e os fundamentos dos testes de software, incluindo objetivos, boas práticas e critérios de aceitação.
- A unidade discute as abordagens caixa-preta e caixa-branca, bem como os principais níveis de teste (unitário, integração, aceitação e regressão).
- O TDD é apresentado como estratégia orientada à qualidade, além dos testes alfa e beta, realizados antes da liberação do software.
- Na sequência, são tratados os fundamentos da gestão de mudanças, com foco em avaliação de impactos, priorização e rastreabilidade.
- A evolução do software é analisada sob a ótica de abordagens ágeis, com entregas incrementais e integração contínua. Por fim, a unidade destaca a gestão de configuração, o versionamento e o uso do GitHub como ferramenta para controle, colaboração e automação, reforçando uma visão integrada de qualidade, controle e evolução contínua do software.

7. Verificação & Validação (V&V)

- A V&V é conduzida por uma participação ativa de equipe, usuários, clientes e demais stakeholders.
- O objetivo da verificação é garantir conformidade com requisitos, padrões e especificações.
 - A verificação é contínua, porque o processo de V&V ocorre em paralelo ao desenvolvimento (CI/CD), reduzindo riscos, aumentando a qualidade e a entrega de valor.
- O objetivo da validação é comprovar que o software atende às necessidades do usuário em ambiente real.
 - As práticas de validação incluem sprint review, prototipação, ATDD e Definition of Done (DoD).
- Nas metodologias ágeis, a validação é contínua e incremental, com feedback rápido a cada entrega.

Nesta seção, serão abordados:

- Verificação do código: depuração e diagnóstico de falhas
- Testes de software: fundamentos, técnicas e práticas
- Testes caixa-preta e caixa-branca

Verificação do Código: Depuração e Diagnóstico de Falhas

- A verificação do código tem o nome técnico de depuração, em que, em metodologias ágeis, deixa de ser um "conserto de última hora" para se tornar parte do DNA da construção do código.

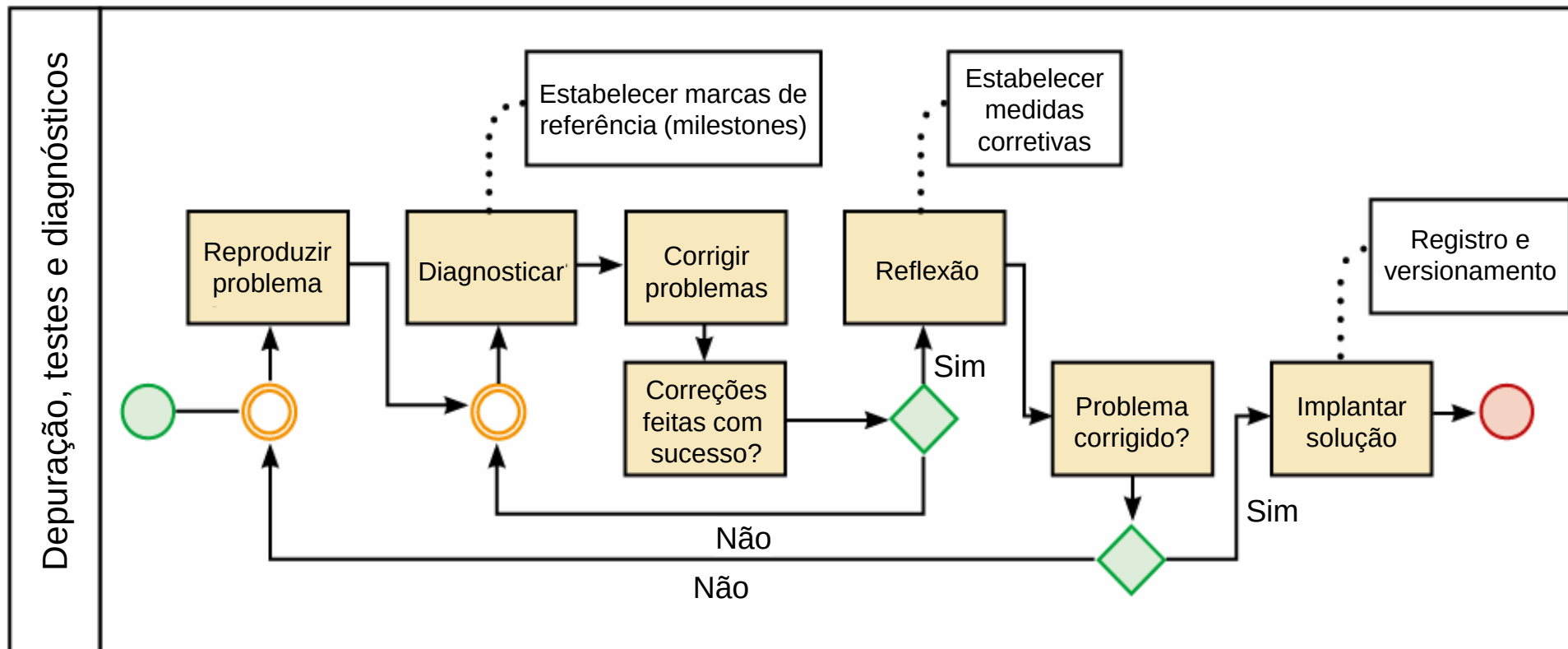
Depuração e colaboração no ciclo ágil

- A atividade depuração (debugging) é contínua de rastreamento e correção de falhas para garantir o funcionamento esperado.
- A transição de modelo sai do processo reativo, tradicional e pós-implementação, e entra no ciclo ágil com depuração colaborativa e incremental.
 - Excelência Técnica – Práticas como TDD, CI, testes automatizados e pair programming previnem e aceleram a identificação de defeitos.
 - Comunicação Ágil – Reuniões diárias (*Daily Scrum*) permitem o compartilhamento imediato de falhas e tomadas de decisão rápidas.

Atividade de Depuração de Falhas (Debug)

Principais atividades de depuração são:

- Reprodução (Reproduce) – Rastreamento para identificação da falha.
- Diagnóstico (Diagnose) – Avaliação da falha, sua causa, gravidade, prioridade e risco.
- Correção (Fix) – Implementação da solução e testes de verificação da correção.
- Reflexão (Reflect) – Aprendizado e medidas aplicadas para evitar a recorrência do problema.



Fluxo das atividades de depuração, testes e diagnósticos do software.
Fonte: Moreno, 2020.

Testes de Software: Fundamentos, Técnicas e Práticas

Nas estratégias e níveis de teste de software, destacam-se:

- Hierarquia de Execução – Evolução dos fundamentos conceituais para os níveis de teste (Unitário, Integração, Sistema) e práticas com usuários finais.
- Validação Dinâmica (SST) – Uso de um conjunto finito de casos de teste para validar o comportamento de um sistema em um domínio de execução infinito.
- Execução e Cobertura – Aplicação de métodos manuais e automatizados para assegurar a confiabilidade e evolução do produto.
- Princípio da Independência – Em sistemas complexos, deve haver separação de papéis: "Quem desenvolve não testa; quem testa não desenvolve", garantindo maior imparcialidade e detecção de falhas.

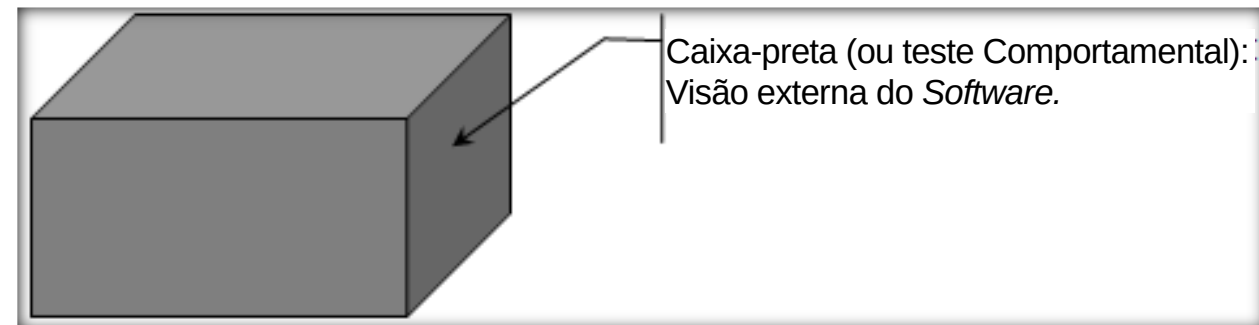
Técnicas Práticas de Testes

Ordem lógica de testes no ciclo de desenvolvimento:

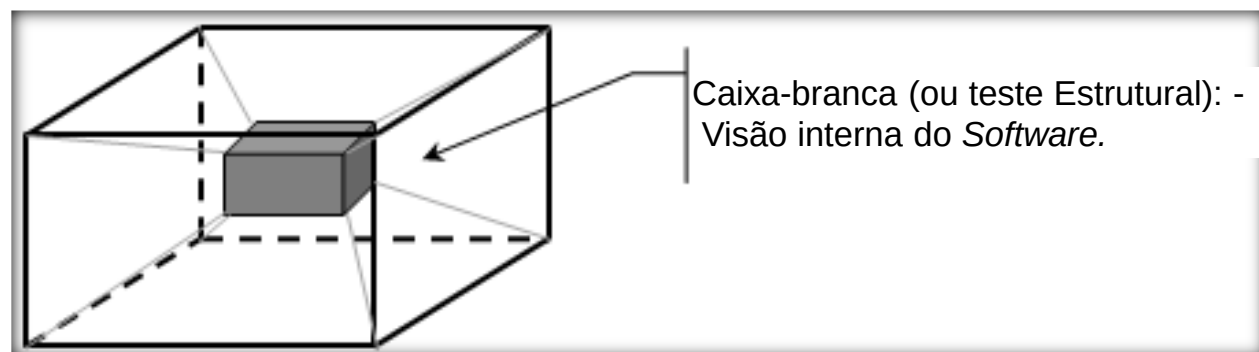
- Testes caixa-preta e caixa-branca – Base metodológica para a elaboração de casos de teste, introduzindo paradigmas que orientam todas as técnicas de testes.
- Testes unitário, de integração, aceitação e regressão – Da menor unidade até o produto final, estruturam a cadeia de verificação, o desenvolvimento de software, constituindo a espinha dorsal (backbone) de testes.
- Prática do desenvolvimento – Organiza-se o ciclo de implementação e testes com o método ágil TDD (Test-Driven Development), redefinindo o papel dos testes unitários ao integrá-los ao ciclo de construção do código.
- Testes alfa e beta – Para validar soluções em ambiente real e com usuários finais.
- Garantia da qualidade – A abordagem Cleanroom prioriza prevenção de defeitos e testes estatísticos de confiabilidade. Essa abordagem combina inspeções, desenvolvimento incremental e testes estatísticos.

Testes Caixa-preta e Caixa-branca

- Os testes caixa-preta e caixa-branca são guiados pelo código-fonte, métricas de software aplicáveis e se realmente estão atendendo aos requisitos com um bom desempenho e segurança.
- Teste caixa-preta (teste comportamental): Identifica as falhas em seu comportamento externo, focado nos requisitos funcionais e conduzidos na interface de software entre o usuário e o ambiente operacional.
- Teste caixa-branca (teste estrutural): Foca nos possíveis erros internos de estrutura dos componentes do sistema de software, hardware, de acesso a ambientes como SGBD e de interconectividade.



Fonte: Moreno, 2023.



Fonte: Moreno, 2023.

Interatividade

Na entrega do produto de software, é comum fazer um teste comportamental do software, com levantamento de casos de teste que possuem referências nos requisitos funcionais. Neste teste, são identificadas falhas e omissões no software. Assinale a alternativa correspondente ao tipo de teste e sua justificativa.

- a) É um teste alfa porque é o primeiro teste a que os usuários serão submetidos nas operações dos requisitos funcionais.
- b) É um teste beta porque se baseia em casos de uso, quando o software entra em operação pelo lado do usuário.
- c) É um teste caixa-branca porque identifica falha no comportamento do software quanto ao requisito funcional.
- d) É um teste caixa-preta porque identifica falha no comportamento externo do software com foco nos requisitos funcionais.
- e) É um teste de IHC porque identifica falha no comportamento dos requisitos funcionais.

Resposta

Na entrega do produto de software, é comum fazer um teste comportamental do software, com levantamento de casos de teste que possuem referências nos requisitos funcionais. Neste teste, são identificadas falhas e omissões no software. Assinale a alternativa correspondente ao tipo de teste e sua justificativa.

- a) É um teste alfa porque é o primeiro teste a que os usuários serão submetidos nas operações dos requisitos funcionais.
- b) É um teste beta porque se baseia em casos de uso, quando o software entra em operação pelo lado do usuário.
- c) É um teste caixa-branca porque identifica falha no comportamento do software quanto ao requisito funcional.
- d) É um teste caixa-preta porque identifica falha no comportamento externo do software com foco nos requisitos funcionais.
- e) É um teste de IHC porque identifica falha no comportamento dos requisitos funcionais.

Teste de Software: Teste Unitário (Unit Test)

- O teste unitário é realizado pelo próprio desenvolvedor durante o processo de codificação e é essencial para identificar erros logo no início do desenvolvimento.

As relações da ISO/IEC 25010 existentes com teste unitário têm o foco em funções isoladas, verificando se cada unidade está correta, contribuindo para avaliar:

- Funcionalidade / adequação funcional – Se cada função implementada corresponde ao requisito especificado.
- Confiabilidade / maturidade – Identifica defeitos antes que ocorram, reduzindo risco de falhas posteriores.
- Manutenibilidade / modularidade – Quanto mais modular o código, mais testável ele é (subcaracterística testabilidade).
- Desempenho (em pequena escala) – Avaliações iniciais de eficiência de métodos específicos.
- Exemplo: Com base no teste caixa-branca, criam-se testes unitários para verificar se a funcionalidade está de acordo com a lógica especificada no caso de teste.

Teste de Software: Integração (Integration Test)

- O teste de integração é realizado para verificar a troca de mensagens entre diferentes módulos ou componentes do software.

Em conformidade com a ISO/IEC 25010, o teste de integração verifica se os componentes funcionam corretamente juntos, contribuindo para avaliar:

- Compatibilidade / interoperabilidade – A integração só funciona quando módulos intercambiam dados corretamente.
- Confiabilidade / maturidade e tolerância a falhas – Garante funcionamento estável entre componentes, principalmente na conectividade.
- Funcionalidade / completude funcional – Valida funções que dependem de outras funções ou variáveis de partes do sistema.
- Segurança / controle de acesso entre módulos – Aplicada em cenários específicos.
- Exemplo: Em fase de teste alfa, um analista de sistemas valida a implantação do componente de software. Para validação do software por parte do cliente, é necessário integrar o componente ao ambiente operacional e fazer os principais testes de integração, como: transferência de dados, interface de hardware e outros.

Teste de Software: Aceitação (Acceptance Test)

- O teste de aceitação, ou teste de validação, ocorre quando o incremento já passou pelos testes unitários e de integração.
- No desenvolvimento ágil, ele está diretamente ligado às histórias do usuário, critérios de aceitação, e testes orientados ao comportamento / aceitação (BDD/ATDD – Behavior-Driven Development / Acceptance Test-Driven Development).
 - Estando na sprint review ou em ciclos de validação contínua, o cliente ou product owner realiza o aceite do incremento entregue.
 - O objetivo é confirmar que o software atende às necessidades do cliente e entrega o valor descrito nos requisitos funcionais e não funcionais.

Em conformidade com a ISO/IEC 25010, o teste de aceitação valida se o sistema atende aos requisitos do cliente e do usuário final, contribuindo para avaliar:

- Qualidade em uso / satisfação do usuário – Testes de aceitação confirmam que o sistema entrega valor ao usuário.
- Funcionalidade / adequação funcional – Se o software atende às necessidades previstas.
- Usabilidade / adequação ao uso – Avalia experiência, mensagens, fluxos e clareza.
- Segurança / integridade e confidencialidade – Em sistemas sensíveis, o aceite inclui validação da política de segurança.

Teste de Software: Regressão (Regression Test)

- Em desenvolvimento ágil, em que mudanças são frequentes e incrementos são liberados rapidamente, o teste de regressão garante que mudanças no código, como correções, melhorias ou novas funcionalidades, não introduzam erros em partes já estáveis do sistema.
- Em ambientes ágeis maduros, os testes de regressão são amplamente automatizados, permitindo verificar rapidamente todo o sistema a cada build gerada.

Em conformidade com a ISO/IEC 25010, o teste de regressão garante que mudanças não impactam em funcionalidades já existentes, contribuindo para avaliar:

- Confiabilidade / estabilidade – Confere se o sistema continua funcionando ao longo das modificações.
 - Manutenibilidade / modificação e testabilidade – Quanto mais fácil adaptar o sistema sem gerar falhas, maior a manutenibilidade.
 - Desempenho (continuidade) – As mudanças não devem degradar a performance geral.
 - Segurança – Alterações não podem introduzir vulnerabilidades.
-
- Exemplo: Em um módulo financeiro de um ERP, um defeito é corrigido durante a sprint e validado por testes de regressão automatizados, garantindo que outras rotinas não sejam afetadas.

Desenvolvimento Guiado por Testes (TDD)

- No design e comportamento no desenvolvimento de software, o FDD e o DDD desenham o mapa do sistema, o BDD e o TDD garantem que cada etapa do ciclo seja segura e validada.

Design e Comportamento: Sinergia e Qualidade

Estrutura e Domínio (FDD + DDD):

- FDD (Feature-Driven) – Organiza o projeto em torno de entregas de valor e escopo.
- DDD (Domain-Driven) – Fornece a base arquitetural para modelar domínios complexos.
- Resultado – Alinhamento total entre o modelo técnico e as necessidades do negócio.

Especificações Executáveis (ATDD + BDD):

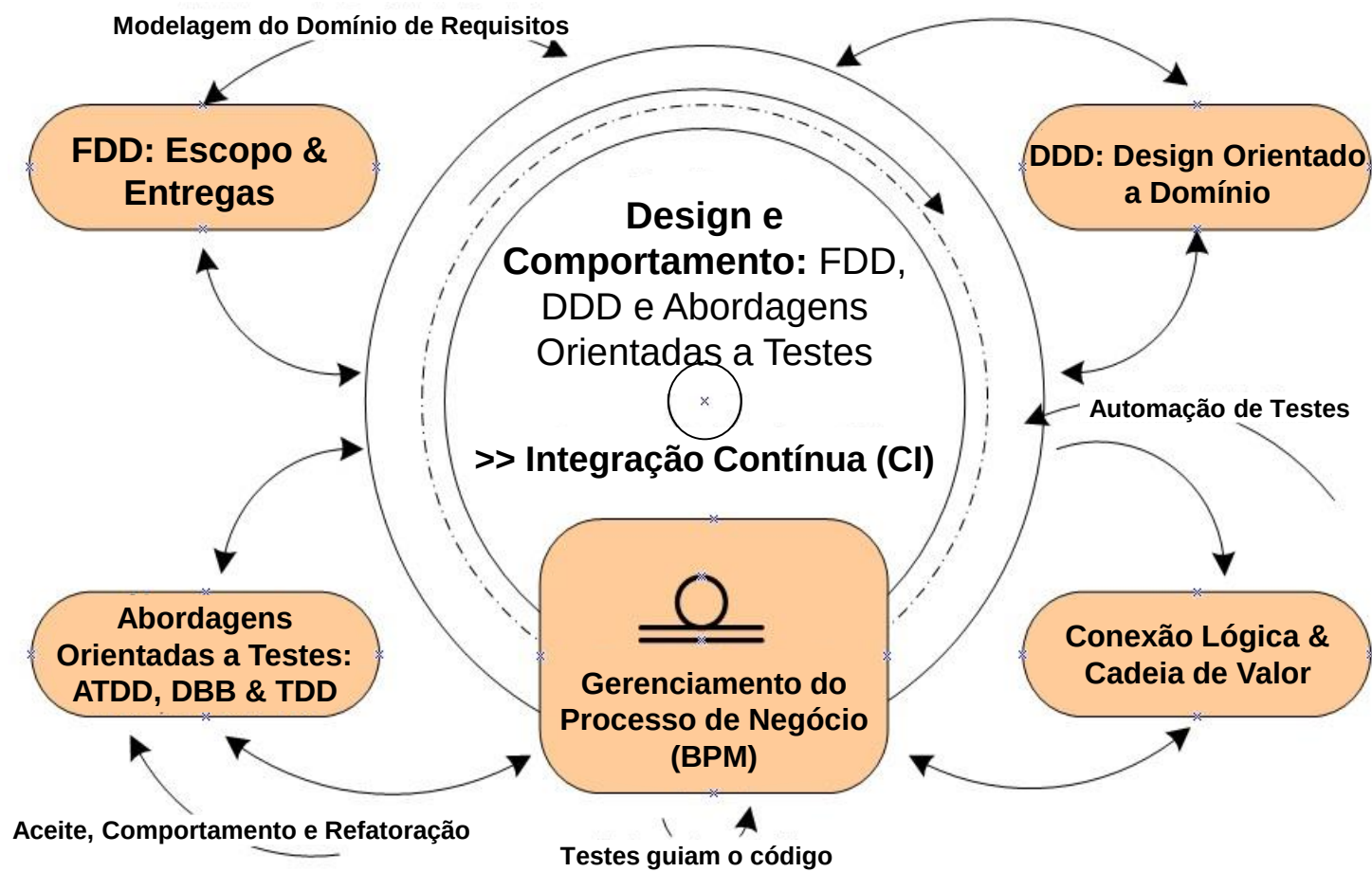
- Tem o foco na colaboração e na perspectiva do usuário.
- Transformam requisitos em testes automatizados que definem o comportamento esperado.

Design de Código e Confiabilidade (TDD):

- Direciona a construção do código-fonte desde o início.
- Garante manutenibilidade e unidades de software livres de defeitos.

Design e Comportamento: FDD, DDD e Abordagens Orientadas a Testes

- A figura mostra os pilares da abordagem ágil, mencionados no design e comportamento do software.
- Sua aplicação integrada é para que qualquer engenheiro de software busque não apenas construir o sistema rapidamente, mas garantir que ele esteja correto, sustentável e fiel ao domínio de negócio.



Design e Comportamento: FDD, DDD e Abordagens Orientadas a Testes.
Fonte: Moreno, 2025.

Testes de Software: Alfa e Beta

No processo de Verificação e Validação (V&V), quando o software é construído especificamente para um cliente, é normal que ele passe por um teste de aceitação por parte do usuário. A melhor estratégia são as dos testes Alfa e Beta:

- O teste Alfa ocorre em ambiente controlado, geralmente na própria empresa desenvolvedora. A equipe de desenvolvimento, a equipe de QA – Quality Assurance (Garantia da Qualidade) e usuários internos (cliente e desenvolvedores) simulam diferentes cenários de uso desde as primeiras etapas do produto, incluindo instalação, configuração, interação e operação.
 - Exemplo: Uma empresa de web design reúne desenvolvedores, designers e representantes do usuário para testar internamente a aplicação web. Neste ambiente, são registradas reações, comentários, dificuldades de uso e sugestões. Esses dados alimentam o backlog e impulsionam melhorias nas próximas iterações.
- O teste Beta – É acompanhado por uma versão release, realizado exclusivamente no habitat do usuário em ambiente descontrolado. Sem a presença dos desenvolvedores, porém monitorado por estes, ao contrário do Teste Alfa.
 - Exemplo: A aplicação web é liberada por release para um grupo diversificado de usuários, permitindo o uso em diferentes sistemas e dispositivos. Os feedbacks e relatórios são automatizados (erros, travamentos, percepções de usabilidade, sugestões e FAQs).

Versões Beta de Software

- A instalação e utilização de uma versão beta de um produto de software deve ser precedida por uma avaliação consciente do risco.
- Os release, por natureza, representam estágios preliminares do ciclo de vida do desenvolvimento, sendo disponibilizados para testes e validação em ambientes reais.

Recomenda-se que o usuário pondere a falta de proficiência técnica para mitigar falhas. Os riscos potenciais incluem, mas não se limitam:

- à instabilidade operacional – Frequentes falhas de sistema, comportamentos inesperados e queda de desempenho.
- à incompatibilidade no ambiente operacional – Problemas de integração e funcionalidade adversa, como configurações de hardware e versões do sistema operacional.
- a vulnerabilidades de segurança – Presença de bugs de segurança não corrigidos que podem expor o sistema a ameaças cibernéticas.
- a limitações de Suporte – Ausência ou redução de suporte técnico, exigindo do usuário a capacidade de diagnosticar e resolver problemas por conta própria.
- ao consumo excessivo de recursos – Subutilização de memória, processamento e armazenamento, impactando a eficiência global do sistema computacional.

Interatividade

As atividades de teste garantem a qualidade do software em diferentes níveis. Leia as descrições a seguir, referentes a essas atividades, e assinale a alternativa que associa corretamente a descrição ao tipo de teste.

- I. Verificação após alterações no código, assegurando que funcionalidades já aprovadas não sejam comprometidas.
 - II. Verificação de componentes isolados, como funções, métodos ou classes.
 - III. Verificação da interação entre módulos integrados do sistema.
 - IV. Verificação final para validar requisitos e regras de negócio sob a perspectiva do usuário.
-
- a) I. Teste Unitário; II. Teste de Integração; III. Teste de Regressão; IV. Teste de Aceitação.
 - b) II. Teste Unitário; III. Teste de Integração; I. Teste de Regressão; IV. Teste de Aceitação.
 - c) III. Teste Unitário; I. Teste de Integração; II. Teste de Regressão; IV. Teste de Aceitação.
 - d) IV. Teste Unitário; II. Teste de Integração; I. Teste de Regressão; III. Teste de Aceitação.
 - e) I. Teste Unitário; III. Teste de Integração; IV. Teste de Regressão; II. Teste de Aceitação.

Resposta

As atividades de teste garantem a qualidade do software em diferentes níveis. Leia as descrições a seguir, referentes a essas atividades, e assinale a alternativa que associa corretamente a descrição ao tipo de teste.

- I. Verificação após alterações no código, assegurando que funcionalidades já aprovadas não sejam comprometidas.
 - II. Verificação de componentes isolados, como funções, métodos ou classes.
 - III. Verificação da interação entre módulos integrados do sistema.
 - IV. Verificação final para validar requisitos e regras de negócio sob a perspectiva do usuário.
-
- a) I. Teste Unitário; II. Teste de Integração; III. Teste de Regressão; IV. Teste de Aceitação.
 - b) II. Teste Unitário; III. Teste de Integração; I. Teste de Regressão; IV. Teste de Aceitação.**
 - c) III. Teste Unitário; I. Teste de Integração; II. Teste de Regressão; IV. Teste de Aceitação.
 - d) IV. Teste Unitário; II. Teste de Integração; I. Teste de Regressão; III. Teste de Aceitação.
 - e) I. Teste Unitário; III. Teste de Integração; IV. Teste de Regressão; II. Teste de Aceitação.

8. Gestão de Mudanças e Evolução do Software

- A gestão de mudanças é um pilar estrutural para manter sistemas funcionais e competitivos em cenários dinâmicos, com requisitos e tecnologias que mudam e a adaptação contínua que garante aderência do software ao seu propósito original.

A estrutura desse pilar se baseia nos seguintes aspectos:

- Processo Estruturado – Correções de erros, análise, planejamento e controle.
- Evolução Inevitável – De acordo com as Leis de Lehman, os sistemas crescem em complexidade e exigem manutenção constante para não se tornarem obsoletos.
- Apoio Metodológico – O uso de metodologias Ágeis e práticas DevOps.
- Resultado – Organizações que gerem mudanças de forma sistêmica respondem melhor ao mercado, promovendo inovação sustentável e garantindo a qualidade final.

Nesta seção, serão abordados:

Fundamentos da Gestão de Mudanças
Evolução, Integração e Tendências do Software
Estratégias estruturais: Top-down e Bottom-up
Gestão de Configuração do Software
MF&T: Controle do código-fonte com GitHub

Fundamentos da Gestão de Mudanças

- A manutenção é uma das maiores peculiaridades da engenharia de software quando comparada a outras áreas. A relação direta com a gestão de mudanças reside no fato de que, no software, a manutenção não é apenas "reparo", mas sim o veículo da sua evolução.
- Dentre as diversas peculiaridades da engenharia de software, diferentemente das demais engenharias, está o processo de manutenção do software.
- Seja na implantação de um sistema de software ou até na simples instalação de um novo aplicativo, a manutenção começa quase que em seguida.
- São problemas de configurações, adaptações ao ambiente operacional, interfaces não operacionais e o maior deles, que são novos requisitos de mudanças por parte do usuário.

Precisa de
manutenção aí?



Fonte: Clipart de biblioteca própria.

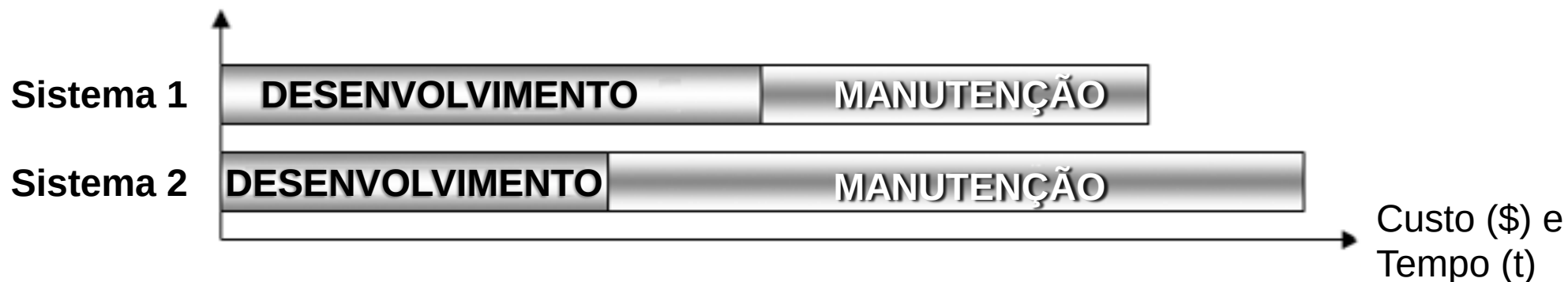
Procedimentos de Manutenção

- O processo de mudança do software ocorre em todo o seu ciclo de vida, para manter o software operacional e adaptado a novas tecnologias.



Das práticas de manutenção, além de se ter o controle de versionamento e testes, a que mais se destaca são mudanças no código-fonte do software.

- Observe o comparativo entre desenvolvimento e manutenção na figura. O Sistema 1, com custo e tempo maiores no desenvolvimento, exigiu no período de manutenção um menor custo e menos tempo. O Sistema 2 foi desenvolvido mais rapidamente com custo e tempo inferiores ao do Sistema 1, porém, exigiu no período de manutenção mais tempo e recursos. O resultado final é que o Sistema 1 obteve sobre o Sistema 2 um menor custo total e em menos tempo.



Tipos de Manutenção

A manutenção de software é o processo geral de mudança em um sistema depois que ele é liberado para uso. O termo geralmente se aplica ao software customizado em que grupos de desenvolvimento separados estão envolvidos antes e depois da liberação (Pressman, 2016).

- A manutenção pode ser corretiva, preventiva ou adaptativa, permitindo ajustes conforme as mudanças tecnológicas e demandas dos usuários.
 - Manutenção corretiva: Manutenção para reparar defeitos no software.
 - Manutenção preventiva: Envolve ações proativas para evitar possíveis problemas futuros no software.
 - Manutenção adaptativa: Para adaptar o software a um novo ambiente operacional, modificar ou fazer acréscimos à funcionalidade do sistema de software.

Manutenção Corretiva: Depuração de Falhas (Debug)

Depuração é a prática de verificação do código-fonte.

- A depuração é a atividade de rastreamento do código, com objetivo de corrigir e reduzir falhas no programa de computador.
- A atividade de verificação é iniciada pelo código e consiste em confrontar o requisito do software com o resultado obtido pelo software, para garantir que o que foi projetado foi construído de acordo.
- Na verificação do código-fonte, faz-se a depuração de falhas (debug), que é a atividade de limpeza ou exclusão de partes indesejáveis.

Debugging (atividade de depuração de falhas)

1. Rastreamento do erro (ou bug): Identificação da causa de um determinado problema.
2. Diagnóstico das causas do erro: Avaliação sobre a gravidade, prioridade, riscos e impactos no comportamento do software.
3. Correção: Implementação e testes para as correções.
4. Reflexão: Medidas corretivas para a solução do problema.

Manutenção Preventiva: Análises, Revisões e Segurança no Código

- Inclui-se aqui a manutenção preventiva por refatoração, que é a prática de reestruturar e melhorar o código-fonte sem alterar seu comportamento externo.
- É o processo de fazer ajustes no código para torná-lo mais legível, eficiente, organizado e sustentável, sem afetar a funcionalidade do software.



Fonte: Clipart de biblioteca própria.

Caso: Manutenção na computação em nuvem

- Na computação em nuvem (cloud computing), a dinâmica de atualizações de código é muito alta.
- Equipes de manutenção realizam verificações regulares no código-fonte para identificar áreas que possam apresentar vulnerabilidades de segurança, corrigindo-as antes que se tornem um problema.

Evolução, Integração e Tendências do Software

- Evolução do Software – Melhoria contínua de desempenho, segurança e usabilidade como motor da transformação tecnológica.
- Integração de Sistemas – Interoperabilidade entre plataformas, serviços e dispositivos; e Suporte a arquiteturas distribuídas e ecossistemas digitais.
- Principais Tendências Tecnológicas – Inteligência Artificial (IA): automação e personalização; Computação em Nuvem: escalabilidade e entrega contínua; Internet das Coisas (IoT): expansão de ecossistemas conectados; e Realidade Virtual/Aumentada: novas formas de interação.
- Tendências em Engenharia de Software – Arquiteturas baseadas em microsserviços; Desenvolvimento orientado a APIs; Automação com IA generativa e observabilidade; e Práticas DevSecOps integradas ao ciclo de vida.
- Impactos Organizacionais – Aumento da eficiência operacional e da experiência do usuário (UX); Necessidade de práticas ágeis, seguras e orientadas a valor; e Competitividade e sustentabilidade dependentes da adaptação contínua.

Interatividade

A ASSERTI é uma pequena empresa desenvolvedora de software, que, na identificação de uma falha do software ou do sistema, usa a engenharia reversa para rastrear o código e diagnosticar a falha, ou seja, rastreia o código partindo da informação e caminha para os dados que geraram a falha. São feitas a correção e a análise do impacto desta mudança no sistema. É correto afirmar que:

- a) a empresa está praticando um processo de Verificação & Validação (V&V) do código.
- b) a empresa utiliza um processo de verificação do código com atividades de depuração de falhas.
- c) é um processo claro de Verificação & Validação (V&V) para garantir que não haverá mudanças posteriores no código.
- d) é um processo de rastreamento do código para validação do software.
- e) este processo é falho porque não avalia a repercussão da correção do problema em outros componentes do software.

Resposta

A ASSERTI é uma pequena empresa desenvolvedora de software, que, na identificação de uma falha do software ou do sistema, usa a engenharia reversa para rastrear o código e diagnosticar a falha, ou seja, rastreia o código partindo da informação e caminha para os dados que geraram a falha. São feitas a correção e a análise do impacto desta mudança no sistema. É correto afirmar que:

- a) a empresa está praticando um processo de Verificação & Validação (V&V) do código.
- b) a empresa utiliza um processo de verificação do código com atividades de depuração de falhas.**
- c) é um processo claro de Verificação & Validação (V&V) para garantir que não haverá mudanças posteriores no código.
- d) é um processo de rastreamento do código para validação do software.
- e) este processo é falho porque não avalia a repercussão da correção do problema em outros componentes do software.

Abordagens Ágeis para Manutenção

- Frameworks como Scrum e Kanban promovem visibilidade do trabalho, priorização dinâmica e ciclos curtos de entrega, permitindo respostas rápidas a defeitos, melhorias e novas demandas de negócio.
 - Os métodos ágeis reforçam a melhoria contínua por meio de feedback frequente e colaboração entre equipes.
 - Práticas técnicas como CI/CD, TDD e automação de testes reduzem riscos e elevam a qualidade do software.
 - A cultura DevSecOps integra desenvolvimento, operações e segurança, viabilizando entregas frequentes e sistemas mais confiáveis.
 - O DevOps integra desenvolvimento e operações para acelerar a entrega e melhorar a qualidade do software. Consulte o link abaixo e veja como a IBM® trabalha.
- IBM®. O que é DevOps?. IBM®. Disponível em: <https://www.ibm.com/br-pt/topics/devops/>. Acesso em: 2 fev. 2026.

Kanban

- Em um cenário do desenvolvimento ágil, a equipe utiliza um quadro Kanban, como mostra a ilustração, dividido em colunas como backlog, A Fazer, Em Progresso, Em Testes, Aguardando Homologação e Concluído. E as cores dos cartões, amplamente utilizadas, seguem níveis de criticidade: Vermelho – Crítico (P1 – Urgente); Laranja – Alto (P2 – Alto impacto); Amarelo – Médio (P3 – Impacto moderado); Verde – Baixo (P4 – Baixo impacto/Melhoria); e Evolutivo/Oportunidade – Azul (P5 – Melhoria futura).

Backlog	A Fazer	Em progresso	Em testes	Aguardando Homologação	Concluído
		 	 		 

Modelo de quadro kanban para acompanhamento de manutenção.
Fonte: Moreno, 2025.

Sprint: Falha na API de autenticação
Identificação: P2
Duração: 2 dias
Responsável: Time (Program.)

Modelo de cartão para quadro kanban.
Fonte: Moreno, 2025.

Evolução de Software

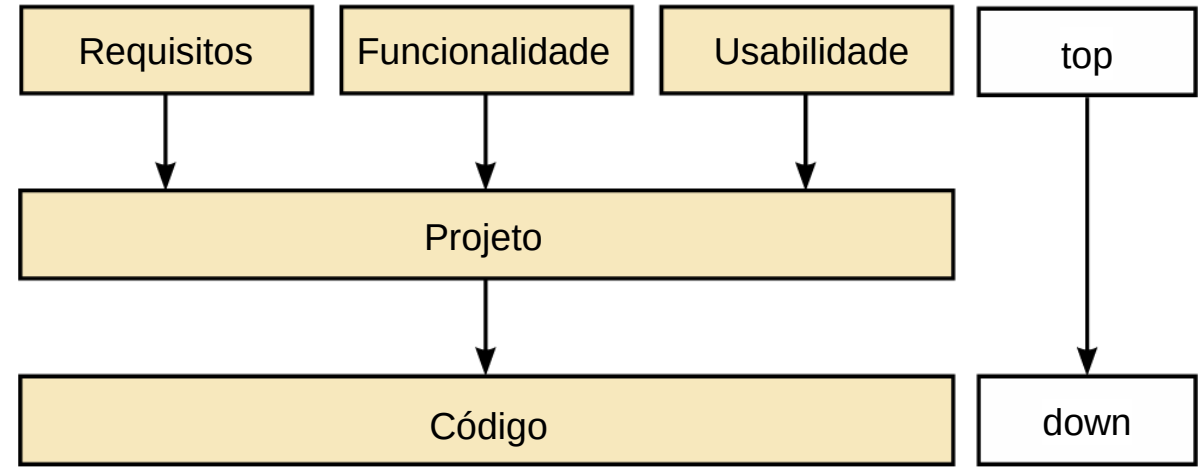
- A evolução do software decorre da necessidade contínua de adaptação às mudanças tecnológicas, de mercado e de comportamento dos usuários, impulsionada por inovações técnicas, integração entre sistemas e busca constante por maior qualidade, desempenho e valor ao negócio.

Ela é voltada para práticas sistematizadas que integram desenvolvimento, manutenção e melhoria contínua em um fluxo único e incremental, como:

- Gestão de backlog e mecanismos de priorização, com a definição clara dos papéis organizacionais, formam a base essencial para essa estrutura de trabalho.
- Integração contínua de defeitos e melhorias impulsionadas pelo DevOpsSec, por arquiteturas modulares e pipelines automatizados.
- Emprego de métricas de fluxo como lead time e cycle time, constituem uma referência para quantificar a eficiência do processo, identificar gargalos e orientar decisões de capacidade e priorização.
- Gestão da dívida técnica e o enquadramento ágil da manutenção contínua despontam como elementos críticos para o futuro da evolução do software.

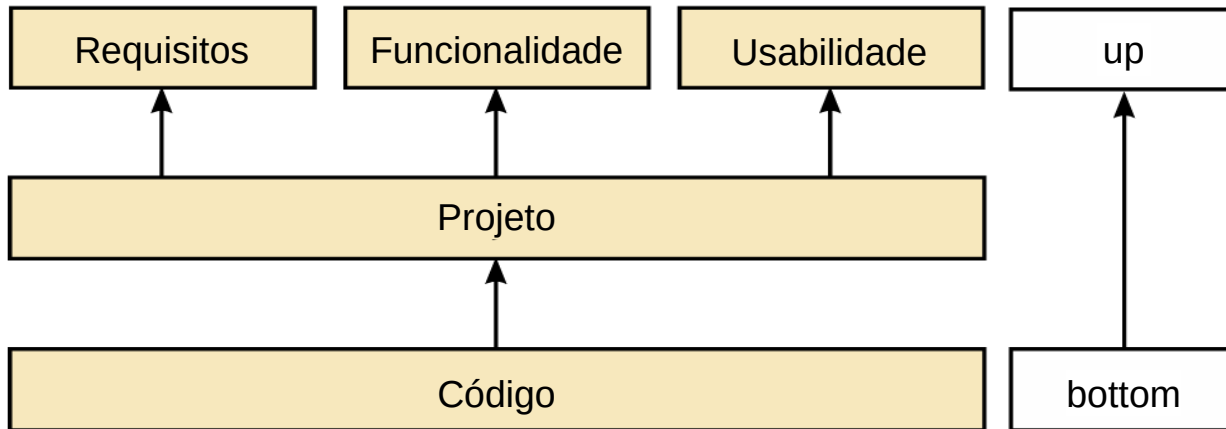
Estratégias Estruturais: Top-down e Bottom-up

- A abordagem top-down verifica a integração dos componentes, começando pelos níveis superiores e de maior valor para o usuário e segue caminhando progressivamente para os módulos subordinados de nível inferior.



A abordagem de teste top-down parte dos níveis superiores, no caso em testes de: requisitos, funcionalidade e usabilidade.

Fonte: Moreno, 2020.



A abordagem de teste bottom-up parte dos níveis inferiores para nível do código.

Fonte: Moreno, 2020.

- A abordagem bottom-up é mais comum no ágil moderno e verifica os componentes, começando pelos níveis mais baixos e granulares (código e módulos) antes de movê-los para cima.

Gestão de Configuração do Software

- A Gestão de Configuração do Software (SCM – Software Configuration Management) controla diversas versões que podem estar em desenvolvimento e uso ao mesmo tempo.
- Se não possuir procedimentos eficazes da gestão de configuração, pode desperdiçar esforço modificando, por exemplo, a versão errada, entregar ao cliente release incorreto ou até esquecer a localização do código-fonte de uma versão ou componente armazenado.
- Indicadas na tabela, Sommerville (2016) sugere quatro principais atividades do gerenciamento de configuração.

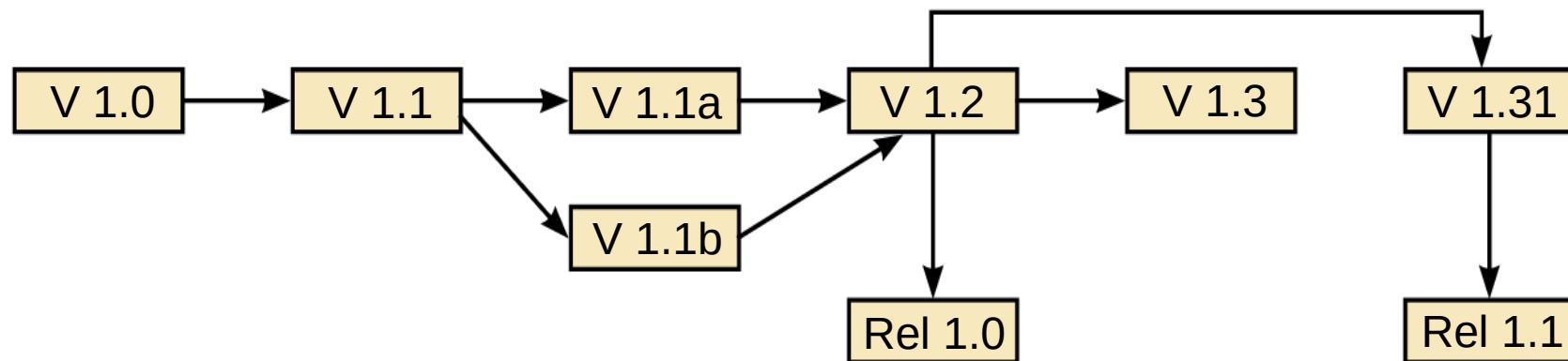
ATIVIDADES	DESCRIÇÃO
1. Controle de versão	Envolve rastrear as múltiplas versões dos componentes do sistema e garantir que alterações feitas por diferentes desenvolvedores não interfiram umas nas outras.
2. Gestão de mudanças	Consiste em registrar solicitações de mudança referentes ao software entre clientes e desenvolvedores, analisar custos e impactos dessas mudanças e decidir se e quando devem ser implementadas.
3. Construção de mudanças	É o processo de reunir componentes de programa, dados e bibliotecas, compilando e vinculando esses elementos para criar um sistema executável.
4. Gestão de releases	Envolve preparar o software para lançamento externo e manter o controle das versões do sistema que foram disponibilizadas para uso pelos clientes.

Versionamento do Software

Técnicas básicas de numeração e indentificação da versão

Numeração de versões	Versão 3.21 – (3) identifica mudança de estrutura, (2) indica inserção de funcionalidade, e (1) indica que houve implementação de uma mudança.
Identificação por atributos	Versão tec01_1: (tec) representa teclado, (01) inserção de mapa de caracteres e (_1) implementação mudança.
Identificação orientada a mudanças	car_04: (car) Carlos requisita uma quarta mudança.

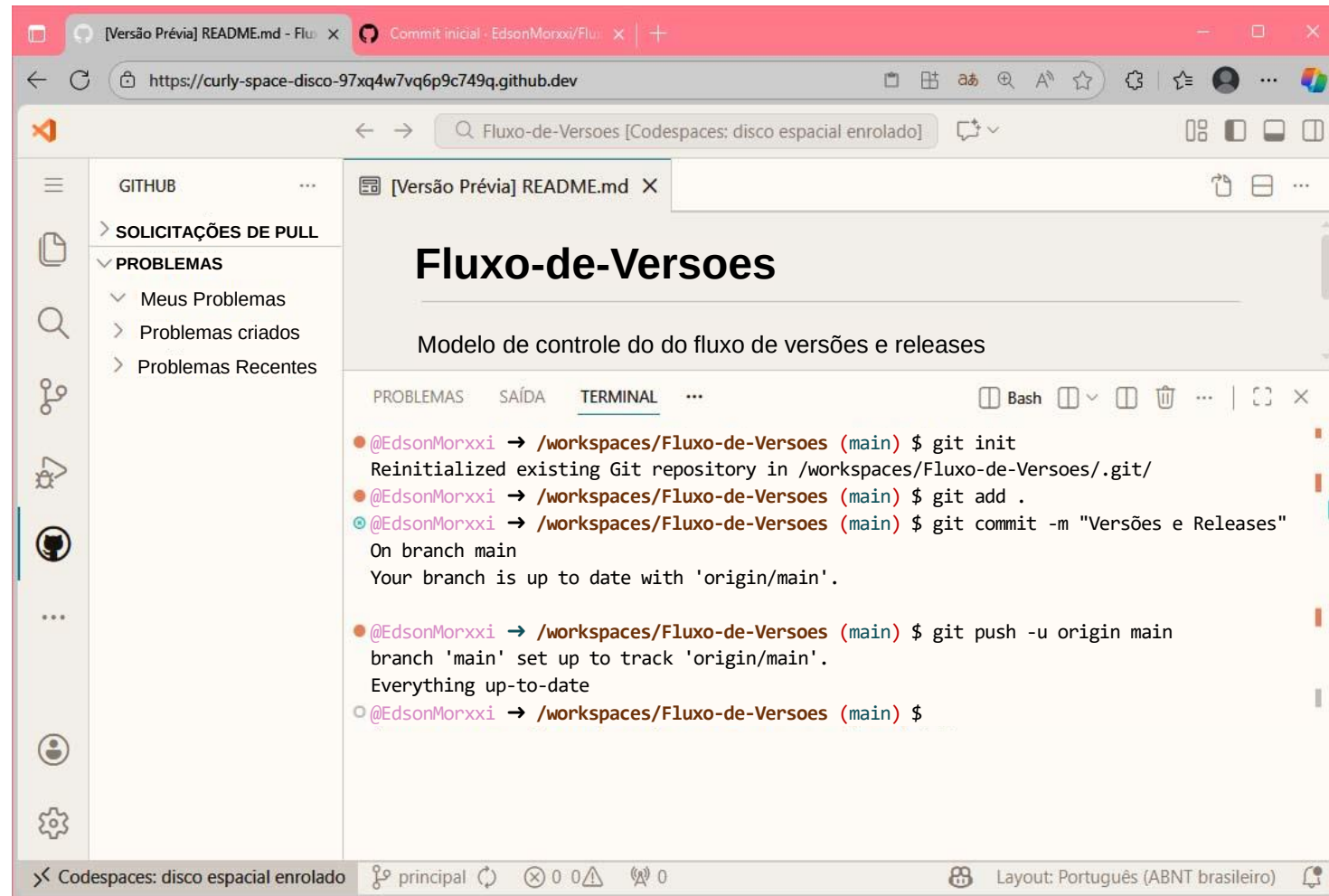
- Versão – São registros de testes ou mudanças e não são liberadas para o cliente.
- Release – É o registro da versão do sistema de software liberada para o cliente.



Modelo de grafo de controle aplicado na numeração de versões e releases.
Fonte: Moreno, 2020.

MF&T: Controle do Código-fonte Com Github

- O GitHub é a maior plataforma de hospedagem de código-fonte e gerenciamento de projetos de software do mundo. Embora seja construído sobre o sistema de controle de versão Git, vai além de armazenar código, atuando como rede social e ferramenta de colaboração completa para desenvolvedores.
- O principal objetivo do GitHub é facilitar a colaboração e o controle de versão de projetos; serve basicamente para hospedagem de repositórios, colaboração distribuída, controle de versão e gerenciamento de projetos.



The screenshot shows a GitHub Codespace interface. The top bar indicates the current workspace is 'Fluxo-de-Versoes [Codespaces: disco espacial enrolado]'. The left sidebar shows the 'GITHUB' menu with options like 'SOLICITAÇÕES DE PULL' and 'PROBLEMAS'. The main area displays the 'Fluxo-de-Versoes' repository with a README file open. Below the README, a terminal window is active, showing the following commands and output:

```
@EdsonMorxxi → /workspaces/Fluxo-de-Versoes (main) $ git init
Reinitialized existing Git repository in /workspaces/Fluxo-de-Versoes/.git/
@EdsonMorxxi → /workspaces/Fluxo-de-Versoes (main) $ git add .
@EdsonMorxxi → /workspaces/Fluxo-de-Versoes (main) $ git commit -m "Versões e Releases"
On branch main
Your branch is up to date with 'origin/main'.

@EdsonMorxxi → /workspaces/Fluxo-de-Versoes (main) $ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
@EdsonMorxxi → /workspaces/Fluxo-de-Versoes (main) $
```

The bottom status bar shows 'Codespaces: disco espacial enrolado', 'principal', and 'Layout: Português (ABNT brasileiro)'.

Interatividade

No processo de gerenciamento de configuração do software, o desenvolvedor faz constantes mudanças no software. Assinale a alternativa que corresponde ao controle das mudanças no software.

- a) A mudança feita no software é controlada pelos logs que o sistema gera. O código gerado pelos logs é utilizado para o controle das mudanças.
- b) A mudança que ocorre no software é especificada na aplicação de patches ou service packs na aplicação de atualização.
- c) As mudanças acompanham a compilação e a ligação dos componentes que são especificadas e registradas nos requisitos funcionais.
- d) O controle das mudanças ocorre no lançamento da aplicação em sua versão beta, em que são registradas todas as observações feitas pelo usuário.
- e) O controle das mudanças ocorre pelo versionamento, sendo as versões os registros das mudanças.

Resposta

No processo de gerenciamento de configuração do software, o desenvolvedor faz constantes mudanças no software. Assinale a alternativa que corresponde ao controle das mudanças no software.

- a) A mudança feita no software é controlada pelos logs que o sistema gera. O código gerado pelos logs é utilizado para o controle das mudanças.
- b) A mudança que ocorre no software é especificada na aplicação de patches ou service packs na aplicação de atualização.
- c) As mudanças acompanham a compilação e a ligação dos componentes que são especificadas e registradas nos requisitos funcionais.
- d) O controle das mudanças ocorre no lançamento da aplicação em sua versão beta, em que são registradas todas as observações feitas pelo usuário.
- e) O controle das mudanças ocorre pelo versionamento, sendo as versões os registros das mudanças.

Referências

- ARKAN. *Descubra como funciona o desenvolvimento de softwares e seus processos*. Arkan System. Disponível em: https://arkansystem.com.br/desenvolvimento-de-softwares_e_seus_processos/. Acesso em: 24 dez. 2020.
- FOURNIER, Roger. *Desenvolvimento e manutenção de sistemas estruturados*. São Paulo: Makron Books, 1994.
- NBR ISO/IEC 9126-1. NBR ISO/IEC 9126-1 – Engenharia de software – Qualidade de produto. Rio de Janeiro: ABNT – Associação Brasileira de Normas Técnicas, 2003.
- NBR ISO/IEC 12207. NBR ISO/IEC 12207 – Tecnologia de informação – Processos de ciclo de vida de software. Rio de Janeiro: ABNT – Associação Brasileira de Normas Técnicas, 1998.
- NBR ISO/IEC 14598. NBR ISO/IEC 14598-1 – Tecnologia de informação – Avaliação de produto de software. Rio de Janeiro: ABNT – Associação Brasileira de Normas Técnicas, 2001.

Referências

- PMBOK/PMI. *Um guia do conhecimento em gerenciamento de projetos*. Guia PMBOK®. 6.ed. EUA: Project Management Institute, 2017
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 8. ed. Rio de Janeiro: McGraw-Hill, 2016.
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021.
- SOMMERVILLE, Ian. *Software engineering*. 10. ed. São Paulo: Pearson Education, 2016.

ATÉ A PRÓXIMA!